Knowledge Organiser — 1.1 Processors, Input, Output & Storage

OCR A Level Computer Science (H446) — Component 01, Section 1.1 (1.1.1 processor, 1.1.2 processor types, 1.1.3 I/O & storage).

Processor components & registers

- **ALU** — arithmetic (+ − × ÷), logic (AND/OR/NOT, compare) and shifts; result → ACC.
- **Control Unit (CU)** — decodes instructions, generates control signals, manages data flow on buses, synchronises with the clock.

Register	Full name	Role
PC	Program Counter	Address of the **next** instruction; incremented each cycle
MAR	Memory Address Register	**Address** of location to read/write
MDR	Memory Data Register	**Data/instruction** just fetched or about to be written
CIR	Current Instruction Register	Holds **current** instruction while decoded/executed
ACC	Accumulator	Stores **immediate result** of ALU calculations

MAR width → max addressable locations (2^n). MDR width ≈ data bus width (throughput/fetch).

Buses

Bus	Direction	Carries
Address	**Unidirectional** (CPU→mem)	Memory address (width → 2^n addressable)
Data	**Bidirectional**	Data/instructions
Control	**Bidirectional**	Control signals (read/write, clock, interrupt, bus request)

Fetch–Decode–Execute cycle

FETCH 1. **PC → MAR** (address copied) 2. Read signal on **control bus**; instruction returns via **data bus → MDR** 3. **PC := PC + 1** (incremented) 4. **MDR → CIR**

DECODE — CU splits CIR into **opcode** (operation) + **operand** (data/address). **EXECUTE** — carry out: ALU op (result → ACC) / load–store memory / **jump** (new address written to PC). Repeat.

Jump/branch = writes new address into PC during execute → causes pipeline branch hazards.

Factors affecting CPU performance

Factor	Effect
Clock speed (Hz)	Pulses/sec sync the CPU; higher = more FDE cycles/sec. Limited by heat & power
Cores	Independent units, each own FDE; multicore runs instructions simultaneously — **only if task parallelisable & software supports it**
Cache	Small very-fast memory **on/near CPU** holding frequent/recent data → fewer slow RAM fetches. Cache miss = slow fetch
Pipelining	Overlaps fetch/decode/execute of consecutive instructions → higher throughput

Cache levels: L1 fastest/smallest (on core) → **L2** larger/slower → **L3** largest/slowest (often shared, still > RAM). Check order: L1→L2→L3→RAM.

Pipeline hazards: branch (flush prefetched) & data (needs in-flight result); fixed by branch prediction / out-of-order execution.

Von Neumann vs Harvard

	Von Neumann	Harvard
Memory	**One** shared (data + instructions)	**Separate** for data & instructions
Buses	Shared	Separate
Access	Can't fetch + read data at once (**bottleneck**)	Simultaneous instruction fetch + data access
Cost	Simpler/cheaper/flexible	Complex/costly
Use	General-purpose PCs/laptops	Embedded, microcontrollers, DSPs

Modern CPUs = hybrid (modified Harvard): unified main memory but split L1 instruction/data cache.

CISC vs RISC

	CISC	RISC
Instructions	Many, complex	Few, simple
Length	Variable, multi-cycle	Fixed, aim 1/cycle
Complexity in	**Hardware**	**Software/compiler**
Pipelining	Harder	Easier
Power	Higher	Lower (battery devices)
Use	x86 desktops	ARM mobile/embedded

GPUs & parallel processing

- **GPU** — thousands of small cores; same operation on large data blocks **in parallel** (data parallelism).

- **Graphics:** render/shade pixels & vertices of 3D scenes simultaneously.
- **GPGPU (non-graphics):** ML / neural-network training, scientific simulation, image/video processing, crypto mining/hashing.
- Poor at sequential, branch-heavy, dependency-laden work (that suits CPU).
- **Multicore:** several cores on one chip, independent simultaneous processing.
- **Parallel processing:** multiple computations at once; split task across cores → faster.
- **Won't parallelise well when:** data dependency (step needs prior result) / coordination overhead / inherently sequential algorithm. Limited by the **serial fraction** that must run sequentially.

Input / output / storage device selection

- No single "right" device — **justify against the scenario**: environment, user/accessibility, accuracy, speed, cost, durability.
- **Input:** optical mouse, keyboard, capacitive touchscreen, barcode scanner, microphone, sensor.
- **Output:** monitor (LCD/LED), laser printer (fast/high-volume/sharp), inkjet (cheap colour/photos/low volume), 3D printer (layer-by-layer), speakers, actuator.
- *Tie each characteristic to a stated requirement (e.g. laser for high-volume office; inkjet for occasional home photos).*

Storage: magnetic / flash / optical

Secondary storage = non-volatile (retains data without power).

Type	How it works	Pros	Cons
Magnetic (HDD, tape)	Magnetised patterns/polarity on spinning platter/tape; R/W head	Huge capacity, low £/GB	Slow (moving parts), fragile, power/noise
Flash/SSD (SSD, USB, SD)	EEPROM; traps electrons in cells; **no moving parts**	Fast, silent, low power, durable, portable	Costlier/GB, finite write cycles
Optical (CD/DVD/Blu-ray)	Laser detects **pits & lands** (reflected light)	Very cheap, portable, long shelf life	Low capacity, slow, scratches, drives rare

Tape = serial access (great cheap archive, bad random). HDD/SSD = direct/random access. Flash uses wear levelling.

RAM vs ROM

	RAM	ROM
Volatility	**Volatile** (lost on power off)	**Non-volatile**
R/W	Read **and** write	Normally read only
Contents	Running programs/data	Bootstrap/BIOS firmware

Virtual storage

- **Virtual memory:** when RAM full, OS uses secondary storage (swap/paging file) as extra RAM; pages moved out/in as needed.

- **Pro:** run more/larger programs than physical RAM allows. **Con:** disk ≪ RAM speed → excess paging = "**disk thrashing**".
- **Virtual/cloud storage:** remote servers over network — access anywhere, scalable, off-site backup; needs internet, ongoing cost, security concerns. *Read question: which "virtual" is meant.*

Must-know definitions

Term	Definition
Register	Small, very fast store inside the CPU holding data during processing
Cache	Small fast memory on/near CPU holding frequently used data/instructions
Pipelining	Overlapping fetch/decode/execute of consecutive instructions for throughput
Core	Independent processing unit able to run its own FDE cycle
Parallel processing	Carrying out multiple computations simultaneously across cores
Volatile	Loses contents when power removed (e.g. RAM)
Virtual memory	Use of secondary storage as extra RAM when physical RAM is full
GPU	Many-core processor for the same operation on large data sets in parallel

Top exam reminders

- **MAR = address, MDR = data** — never swap them.
- In the fetch trace, **increment the PC** AND **copy MDR → CIR** (both often missed).
- **Address bus = unidirectional**; data & control buses = bidirectional.
- "More cores = faster" **only if task parallelises and software supports it** — always add the caveat.
- **Cache is separate fast memory on/near the CPU, not part of RAM.** Flash/SSD is **non-volatile** (unlike RAM).
- For "choose a device/storage" questions, **link each characteristic to the scenario** and actually **compare** ("whereas") — generic pros/cons caps marks at AO1.